

Approximation Algorithms

Linear Programming and Integer Programming

Linear Programming (LP) Problem

- The maximization or minimization of some quantity is the objective in all linear programming problems.
- All LP problems have constraints that limit the degree to which the objective can be pursued.
- If both the objective function and the constraints are linear, the problem is referred to as a linear programming problem.

Linear Programming

$$\text{Max/min} \quad z = c_1x_1 + c_2x_2 + \dots + c_nx_n$$

subject to:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n (\leq, =, \geq) b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n (\leq, =, \geq) b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n (\leq, =, \geq) b_m \end{cases}$$

x_j = variables

b_i = constraint levels

c_j = objective function coefficients

a_{ij} = constraint coefficients

Solutions

- A solution is a value assignment to all variables.
- A feasible solution satisfies all the problem's constraints.
- An optimal solution is a feasible solution that results in the largest possible objective function value when maximizing (or smallest when minimizing).

Problem Formulation

- Problem formulation or modeling is the process of translating a verbal statement of a problem into a mathematical statement.

LP Formulation: Example

RESOURCE REQUIREMENTS			
PRODUCT	<i>Labor</i> (hr/unit)	<i>Clay</i> (lb/unit)	<i>Revenue</i> (\$/unit)
Bowl	1	4	40
Mug	2	3	50

There are 40 hours of labor and 120 pounds of clay available each day

Question: How many bowls and mugs should we produce to maximize the revenue?

Decision variables

x_1 = number of bowls to produce

x_2 = number of mugs to produce

LP Formulation: Example

Maximize $Z = \$40 x_1 + 50 x_2$

Subject to

$$x_1 + 2x_2 \leq 40 \text{ hr} \quad (\text{labor constraint})$$

$$4x_1 + 3x_2 \leq 120 \text{ lb} \quad (\text{clay constraint})$$

$$x_1, x_2 \geq 0$$

Solution is $x_1 = 24$ bowls $x_2 = 8$ mugs

Revenue = \$1,360

Approaches for Solving LP

- Simplex Algorithm (1947)
- Ellipsoid Method (1972, polynomial-time)
- Packages (e.g., CPLEX)

Integer Linear Programming

- Basically linear programming but all variables must be integers.
- NP hard.

Linear Programming and Randomized Rounding

MAX SAT

- Given n boolean variables $x_1, x_2, \dots, x_n$, m clauses $C_i, i = 1, \dots, m$ over those variables and a weight $w_i \geq 0$ for each clause, the *MAX SAT* problem is to find a truth assignment for the variables that maximizes the total weight of the clauses satisfied.

Integer Programming for MAX SAT

- For every variable x_i , define $y_i \in \{0,1\}$ denote whether it is assigned true.
- For every clause C_j , define $z_j \in \{0,1\}$ to denote whether it is satisfied.
- For every clause C_j let I_j^+ be the set of indices of variables that are not negated in C_j and I_j^- the set of indices of variables that are negated.
 - For example, when $C_j = x_1 \vee \bar{x}_2 \vee x_3 \vee x_4 \vee \bar{x}_5$ then $I_j^+ = \{1,3,4\}$ and $I_j^- = \{2,5\}$.
- Now construct the problem:

Integer Programming for MAX SAT

$$\begin{aligned} &\text{Maximize } \sum_j w_j z_j \\ &\text{subject to conditions} \\ &\quad \forall j, \sum_{i \in I_j^+} y_i + \sum_{i \in I_j^-} (1 - y_i) \geq z_j \\ &\quad \forall j, z_j \in \{0, 1\} \\ &\quad \forall i, y_i \in \{0, 1\} \end{aligned}$$

- Construct a correspondence between truth assignment S and the values of y_i by setting $y_i = 1$ if and only if $x_i = T$ and $y_i = 0$ iff $x_i = F$.
- Assignment S satisfies C_j if and only if $\sum_{i \in I_j^+} y_i + \sum_{i \in I_j^-} (1 - y_i) \geq 1$.
- For this fixed assignment the objective function $\sum_j w_j z_j$ is maximized if we set $z_j = 1$ for every C_j that is satisfied (the unsatisfied C_j have $z_j = 0$).
- This implies that the objective function $\sum_j w_j z_j$ is maximized when the y_i are assigned values corresponding to a MAX-SAT assignment.

Integer Programming for MAX SAT

- How does the integer programming help? Solving integer linear programs is NP-Hard.
- But, if we relax the problem and no longer require the y_i and z_j to be integers we get a regular linear program. Linear programs can be solved in polynomial time using off-the-shelf software.

Relaxed LP

Maximize $\sum_j w_j z_j$

Subject to conditions

$$\forall j, \sum_{i \in I_j^+} y_i + \sum_{i \in I_j^-} (1 - y_i) \geq z_j$$

$$\forall j, 0 \leq z_j \leq 1$$

$$\forall i, 0 \leq y_i \leq 1$$

Relaxed LP for MAX SAT

We use the notation y^*, z^* to denote the variables in the optimal solution for the relaxed LP. Note that if $z_{LP} = \sum_j w_j z_j^*$ is the optimum (maximum) calculated in the relaxed linear program and OPT is the optimal solution for MAX SAT then

$$z_{LP} \geq OPT$$

Randomized Rounding

- Rounding: covert the real-valued solutions to integer solutions.
 - Can be considered as a randomized algorithm.

Randomized Rounding
Solve the Relaxed Linear Program
Calculate (y^*, z^*)
for $i = 1$ to n do
 Flip a y_i^* -biased coin
 If Heads set $x_i = T$
 else set $x_i = F$

- Theorem: Let W be the weight of the assignment created by Randomized Rounding. Then

$$E(W) \geq \left(1 - \frac{1}{e}\right) OPT$$

- where $1 - \frac{1}{e} \approx 0.632$.

Randomized Rounding: Proof

- The proof will need two facts:

1. If $a_i \geq 0$ then

$$\forall k, (a_1 a_2 \dots a_k)^{\frac{1}{k}} \leq \frac{1}{k} (a_1 + a_2 + \dots + a_k).$$

2. If function $f(x)$ is concave on $[l, u]$, $f(l) \geq al + b$, and $f(u) \geq au + b$ then

$$\forall x \in [l, u], f(x) \geq ax + b.$$

Randomized Rounding: Proof

Apply fact 2 to

$$f(x) = 1 - \left(1 - \frac{x}{k}\right)^k$$

on $[0,1]$. This $f()$ is concave on $[0,1]$,

$$f(0) = 0, f(1) = 1 - \left(1 - \frac{1}{k}\right)^k,$$

so for $x \in [0,1]$, $f(x) \geq \left(1 - \left(1 - \frac{1}{k}\right)^k\right)x$.

This implies that

$$\forall x \in [0,1], 1 - \left(1 - \frac{x}{k}\right)^k \geq \left(1 - \left(1 - \frac{1}{k}\right)^k\right)x.$$

- Start the proof by considering a clause C_j that has only all positive variables, e.g., $C_j = x_1 \vee x_2 \vee \cdots \vee x_k$. Since $I_j^- = \emptyset$ the LP constraint for C_j was $\sum_{i=1}^k y_i^* \geq z_j^*$.

- From the algorithm $\Pr(x_i = T) = y_i^*$ so

$$\begin{aligned} \Pr(C_j \text{ is satisfied}) &= 1 - \prod_{i=1}^k (1 - y_i^*) \\ &\geq 1 - \left(\frac{k - \sum_{i=1}^k y_i^*}{k} \right)^k \\ &\geq 1 - \left(1 - \frac{z_j^*}{k} \right)^k \\ &\geq \left(1 - \left(1 - \frac{1}{k} \right)^k \right) z_j^* \end{aligned}$$

- The first inequality comes from fact 1, the second from $\sum_{i=1}^k y_i^* \geq z_j^*$, and the third from fact 2.

- Now consider a clause C_j of the form

$$C_j = x_1 \vee x_2 \dots \vee x_l \vee \bar{x}_{l+1} \dots \vee \bar{x}_k.$$

- The LP constraint for C_j was $\sum_{i=1}^l y_i^* + \sum_{i=l+1}^k (1 - y_i^*) \geq z_j^*$. Now

$$\begin{aligned} \Pr(C_j \text{ is satisfied}) &= 1 - \prod_{i=1}^l (1 - y_i^*) \prod_{i=l+1}^k y_i^* \\ &\geq 1 - \left(\frac{l - \sum_{i=1}^l y_i^* + \sum_{i=l+1}^k y_i^*}{k} \right)^k \\ &\geq 1 - \left(1 - \frac{z_j^*}{k} \right)^k \\ &\geq \left(1 - \left(1 - \frac{1}{k} \right)^k \right) z_j^* \end{aligned}$$

- The only new aspect of this derivation is in the second equality. It comes from the fact that

$$\begin{aligned} l - \sum_{i=1}^l y_i^* + \sum_{i=l+1}^k y_i^* &= k - \sum_{i=1}^l y_i^* - \sum_{i=l+1}^k (1 - y_i^*) \\ &\leq k - z_j^* \end{aligned}$$

- We have actually just proven that if C_j is a clause with l_j variables then

$$\Pr(C_j \text{ is satisfied}) \geq \left(1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right) z_j^*.$$

- Thus

$$\begin{aligned} E(W) &= \sum_j w_j \Pr(C_j \text{ is satisfied}) \\ &\geq \sum_j w_j \left(1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right) z_j^* \\ &\geq \min_j \left(1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right) \sum_j w_j z_j^* \\ &\geq \left(1 - \frac{1}{e}\right) OPT \end{aligned}$$

- which is what we have been attempting to prove.
- We used the facts that $\forall x > 0, \left(1 - \frac{1}{x}\right)^x < e^{-1}$ and that $\sum_j w_j z_j^* \geq OPT$.

MAX-SAT: Best of Two

- Consider two different approaches to approximating MAX-SAT:

- Random MAX-SAT

$$E(W) \geq \frac{OPT}{2}$$

- This chose a random truth assignment using a fair coin. For clause C_j with length l_j

$$\Pr(C_j \text{ is satisfied}) = 1 - 2^{-l_j}$$

- Randomized Rounding

$$E(W) \geq \left(1 - \frac{1}{e}\right) OPT \approx 0.632 \cdot OPT$$

- Finds solution (y^*, x^*) to relaxed linear program. For clause C_j with length l_j

$$\Pr(C_j \text{ is satisfied}) \geq \left(1 - \left(1 - \frac{1}{l_j}\right)^{l_j}\right) z_j^*$$

- Notice that Random MAX-SAT is "good" for long clauses while Randomized Rounding is "good" for short clauses.

MAX-SAT: Best of Two

- How to combine the two to get an even better approximation?
- The Best of Two algorithm is to run Random MAX-SAT to get assignment x^1 with weight W_1 and to also run Randomized Rounding to get assignment x^2 with weight W_2 . Then compare W_1 and W_2 . If $W_1 > W_2$ return x^1 , else return x^2 . Let W be the weight of the returned assignment.
- Theorem: $E(W) \geq \frac{3}{4} OPT$.

Linear Programming and Randomized Rounding for MAX SAT

MAX-SAT $\rightarrow$ Integer Linear Programming

ILP $\rightarrow$ Relaxed LP
with solution (y^*, z^*)
and known max ZLP

(Rounding) Chose a random assignment based on y^*

Show that $E(W) \geq \left(1 - \frac{1}{e}\right) OPT$

Linear Programming and Randomized Rounding

Problem $\rightarrow$ Integer Linear Programming

ILP $\rightarrow$ Relaxed LP
with solution (x^*)
and known max

(Rounding) Chose a random solution based on (x^*)

Show the approximation ratio of the solution

(For some problems) Show the probability that the solution is correct

Randomized Rounding for Set Cover

- Recall the problem: Let $U = \{u_1, \dots, u_n\}$ be a set and $\mathcal{F} = \{F_1, F_2, \dots, F_m\}$ be a family of subsets of U such that $U = \bigcup_i F_i$.
- The Set Cover problem is to find a minimum-size subset $\mathcal{C} \subseteq \mathcal{F}$ that covers U , i.e., $U = \bigcup_{F \in \mathcal{C}} F$. In what follows OPT is the size of a minimum set cover.
- We will see how to use Randomized Rounding to find a subset $\mathcal{C} \subseteq \mathcal{F}$ that has the following properties.

$$E(|\mathcal{C}|) \leq 2 \ln n \cdot OPT$$

$$\Pr(\mathcal{C} \text{ is a set cover}) \geq 1 - \frac{1}{n}$$

Randomized Rounding for Set Cover

- Step 1: Construct the integer linear program
 - Define binary variable x_i such that F_i is included in $\mathcal{C}$ iff $x_i = 1$
 - The solution is expressed as $\mathcal{C}(x) = \{F_i : x_i = 1\}$
 - The cardinality of $\mathcal{C}(x)$ is given by $\sum_i x_i$
 - $\mathcal{C}(x)$ is a cover if for each u_j we have $\sum_{u_j \in F_i} x_i \geq 1$, i.e., u_j is covered

ILP for Set Cover

Minimize $\sum_i x_i$

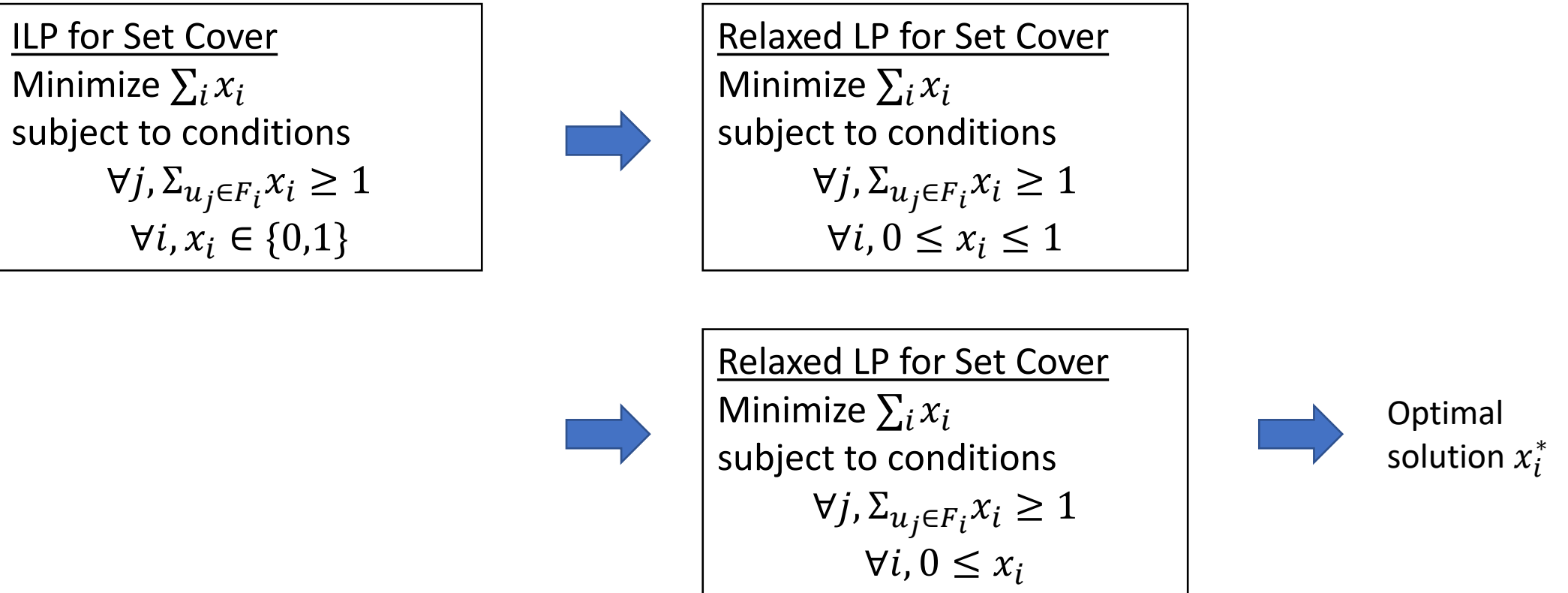
subject to conditions

$$\forall j, \sum_{u_j \in F_i} x_i \geq 1$$

$$\forall i, x_i \in \{0,1\}$$

Randomized Rounding for Set Cover

- Step 2: Relax the ILP to a standard LP



Randomized Rounding for Set Cover

- Step 3: Rounding

Randomized Rounding for Set Cover

Set $\mathcal{C} = \emptyset$.

for $i = 1$ to n do

 Flip a x_i^* -biased coin

 If “heads” put F_i into $\mathcal{C}$.

- Note that

$$E(|\mathcal{C}|) = \sum_i x_i^* \leq OPT$$

- Our major problem is that nothing says that $\mathcal{C}$ must be a set cover.

Randomized Rounding for Set Cover

- Theorem: For each $u_j \in U$

$$\Pr(\mathcal{C} \text{ covers } u_j) \geq 1 - \frac{1}{e}$$

- Proof: Suppose that u_j is contained in k sets of $\mathcal{F}$. W.L.O.G. we may assume that these are $F_1, F_2, \dots, F_k$. From the linear program we have that

$$\sum_{i=1}^k x_i^* \geq 1$$

- Now

$$\begin{aligned} \Pr(\mathcal{C} \text{ does not cover } u_j) &= \Pr(\text{None of } F_1, F_2, \dots, F_k \text{ are in } \mathcal{C}) \\ &= \prod_{i=1}^k (1 - x_i^*) \\ &\leq \left(\frac{\sum_{i=1}^k (1 - x_i^*)}{k} \right)^k \\ &\leq \left(1 - \frac{1}{k} \right)^k \leq \frac{1}{e} \end{aligned}$$

- The proof follows by taking the complement.

Randomized Rounding for Set Cover

- Theorem: $\Pr(\mathcal{C} \text{ is a set cover}) \geq \left(1 - \frac{1}{e}\right)^n$  Can be very small when n is large
- Meanwhile, $E(|\mathcal{C}|) \leq OPT$
- Can we trade off the approximation ratio for this probability?
- Yes, run the randomized rounding multiple times.

Randomized Rounding for Set Cover

- Now Run the Randomized Rounding algorithm $K = \lceil 2\ln n \rceil$ times, constructing K collections $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K$ and then setting

$$\mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2 \cup \dots \cup \mathcal{C}_K$$

- Note that $E(|\mathcal{C}|) \leq \sum_{i=1}^K E(|\mathcal{C}_i|) \leq \lceil 2\ln n \rceil OPT$.
- For any given j

$$\Pr(\mathcal{C} \text{ does not cover } u_j) \leq \left(\frac{1}{e}\right)^K \leq \left(\frac{1}{e}\right)^{2\ln n} = \frac{1}{n^2}.$$

- The only way that $\mathcal{C}$ can not be a cover is if one of the u_j is not covered by $\mathcal{C}$ so

$$\Pr(\mathcal{C} \text{ is not a set cover}) \leq \sum_j \Pr(\mathcal{C} \text{ does not cover } u_j) \leq n \frac{1}{n^2} = \frac{1}{n}$$

- Thus

$$\Pr(\mathcal{C} \text{ is a set cover}) \geq 1 - \frac{1}{n}$$

Approximation Algorithms for Set Cover

- Greedy Set Cover: an H_n -approximation algorithm.
- Randomized Rounding for Set Cover: a $2 \ln n$ -approximation algorithm with probability $1 - \frac{1}{n}$ to produce correct solution.

